

[Home](#) > [Tutorials](#) > [Machine Learning](#)

Introduction to k-Means Clustering with scikit-learn in Python

In this tutorial, learn how to apply k-Means Clustering with scikit-learn in Python

Updated Mar 10, 2023 · 21 min read

Kevin Babitz

Data Science writer | Senior Technical Marketing Analyst at Wayfair | MSE in Data Science at University of Pennsylvania

TOPICS

Machine Learning

Data Science

Python



Introduction

In this tutorial, you will learn about k-means clustering. We'll cover:

- How the k-means clustering algorithm works
- How to visualize data to determine if it is a good candidate for clustering
- A case study of training and tuning a k-means clustering model using a real-world California housing dataset.

Note that this should not be confused with k-nearest neighbors, and readers wanting that should go to k-Nearest Neighbors (KNN) Classification with scikit-learn in Python instead.

This is useful to know as k-means clustering is a popular clustering algorithm that does a good job of grouping spherical data together into distinct groups. This is very valuable as

both an analysis tool when the groupings of rows of data are unclear or as a feature-engineering step for improving supervised learning models.

We expect a basic understanding of Python and the ability to work with pandas Dataframes for this tutorial.

Develop AI Applications

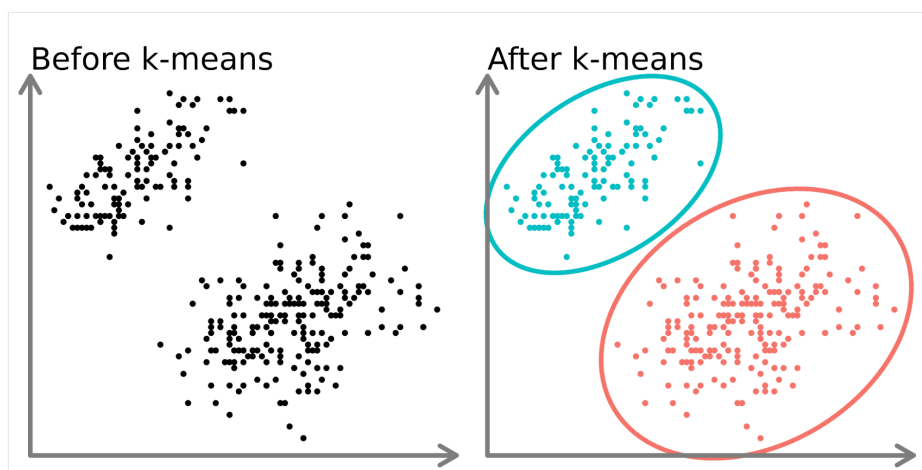
Learn to build AI applications using the OpenAI API.

Start Upskilling for Free

An Overview of K-Means Clustering

Clustering models aim to group data into distinct “clusters” or groups. This can both serve as an interesting view in an analysis, or can serve as a feature in a supervised learning algorithm.

Consider a social setting where there are groups of people having discussions in different circles around a room. When you first look at the room, you just see a group of people. You could mentally start placing points in the center of each group of people and name that point as a unique identifier. You would then be able to refer to each group by a unique name to describe them. This is essentially what k-means clustering does with data.



In the left-hand side of the diagram above, we can see 2 distinct sets of points that are unlabeled and colored as similar data points. Fitting a k-means model to this data (right-hand side) can reveal 2 distinct groups (shown in both distinct circles and colors).

In two dimensions, it is easy for humans to split these clusters, but with more dimensions, you need to use a model.

The Dataset

In this tutorial, we will be using California housing data from Kaggle ([here](#)). We will use location data (latitude and longitude) as well as the median house value. We will cluster the houses by location and observe how house prices fluctuate across California. We save the dataset as a csv file called ‘housing.csv’ in our working directory and read it using pandas .

```
import pandas as pd
```



```
home_data = pd.read_csv('housing.csv', usecols = ['longitude', 'latitude', 'median_house_value'])  
home_data.head()
```

 Explain code

POWERED BY  datalab

	longitude	latitude	median_house_value
0	-122.23	37.88	452600.0
1	-122.22	37.86	358500.0
2	-122.24	37.85	352100.0
3	-122.25	37.85	341300.0
4	-122.25	37.85	342200.0

The data include 3 variables that we have selected using the `usecols` parameter:

- **longitude:** A value representing how far west a house is. Higher values represent houses that are further West.
- **latitude:** A value representing how far north a house is. Higher values represent houses that are further north.
- **median_house_value:** The median house price within a block measured in USD.

k-Means Clustering Workflow

Like other Machine Learning algorithms, k-Means Clustering has a workflow (see [A Beginner's Guide to The Machine Learning Workflow](#) for a more in depth breakdown of the Machine learning workflow).

In this tutorial, we will focus on collecting and splitting the data (in data preparation) and hyperparameter tuning, training your model, and assessing model performance (in modeling). Much of the work involved in unsupervised learning algorithms lies in the hyperparameter tuning and assessing performance to get the best results from your model.

Visualize the Data

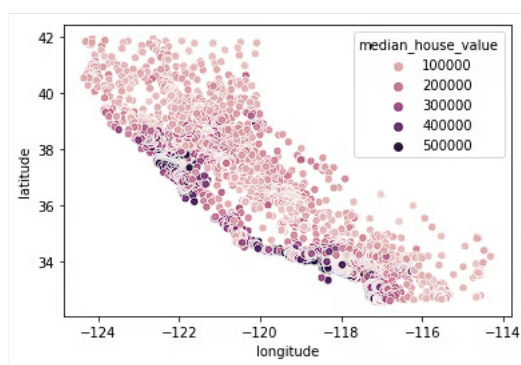
We start by visualizing our housing data. We look at the location data with a heatmap based on the median price in a block. We will use Seaborn to quickly create plots in this tutorial (see our [Introduction to Data Visualization with Seaborn course](#) to better understand how these graphs are being created).

```
import seaborn as sns
```

```
sns.scatterplot(data = home_data, x = 'longitude', y = 'latitude', hue = 'median_
```

 Explain code

POWERED BY  datalab



We see that most of the expensive houses are on the west coast of California with different areas that have clusters of moderately priced houses. This is expected as typically waterfront properties are worth more than houses that are not on the coast.

Clusters are often easy to spot when you are only using 2 or 3 features. It becomes increasingly difficult or impossible when the

Normalizing the Data

When working with distance-based algorithms, like k-Means Clustering, we must normalize the data. If we do not normalize the data, variables with different scaling will be weighted differently in the distance formula that is being optimized during training. For example, if we were to include price in the cluster, in addition to latitude and longitude, price would have an

outsized impact on the optimizations because its scale is significantly larger and wider than the bounded location variables.

We first set up training and test splits using `train_test_split` from `sklearn`.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(home_data[['latitude', 'longi
```

 Explain code

POWERED BY  datacamp

Next, we normalize the training and test data using the `preprocessing.normalize()` method from `sklearn`.

```
from sklearn import preprocessing

X_train_norm = preprocessing.normalize(X_train)
X_test_norm = preprocessing.normalize(X_test)
```

 Explain code

POWERED BY  datacamp

Fitting and Evaluating the Model

For the first iteration, we will arbitrarily choose a number of clusters (referred to as `k`) of 3. Building and fitting models in `sklearn` is very simple. We will create an instance of `KMeans`, define the number of clusters using the `n_clusters` attribute, set `n_init`, which defines the number of iterations the algorithm will run with different centroid seeds, to “auto,” and we will set the `random_state` to 0 so we get the same result each time we run the code. We can then fit the model to the normalized training data using the `fit()` method.

```
from sklearn import KMeans

kmeans = KMeans(n_clusters = 3, random_state = 0, n_init='auto')
kmeans.fit(X_train_norm)
```

 Explain code

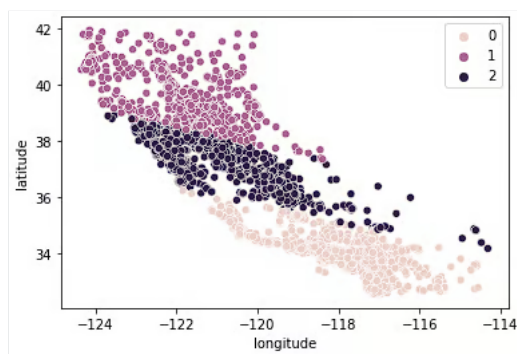
POWERED BY  datacamp

Once the data are fit, we can access labels from the `labels_` attribute. Below, we visualize the data we just fit.

```
sns.scatterplot(data = X_train, x = 'longitude', y = 'latitude', hue = kmeans.labels_)
```

 Explain code

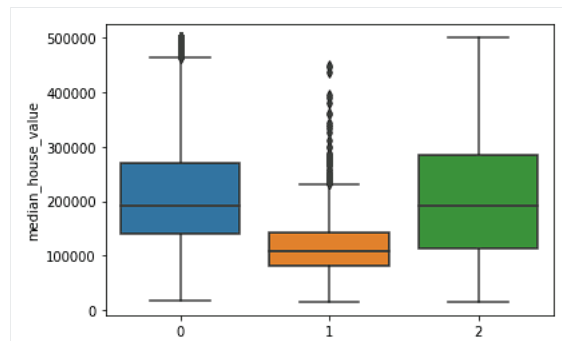
POWERED BY  datacamp



We see that the data are now clearly split into 3 distinct groups (Northern California, Central California, and Southern California). We can also look at the distribution of median house prices in these 3 groups using a boxplot.

```
sns.boxplot(x = kmeans.labels_, y = y_train['median_house_value'])
```



[🔗 Explain code](#)
POWERED BY  datalab

We clearly see that the Northern and Southern clusters have similar distributions of median house values (clusters 0 and 2) that are higher than the prices in the central cluster (cluster 1).

We can evaluate performance of the clustering algorithm using a Silhouette score which is a part of `sklearn.metrics` where a lower score represents a better fit.

```
from sklearn.metrics import silhouette_score

silhouette_score(X_train_norm, kmeans.labels_, metric='euclidean')
```


[🔗 Explain code](#)
POWERED BY  datalab

Since we have not looked at the strength of different numbers of clusters, we do not know how good of a fit the $k = 3$ model is. In the next section, we will explore different clusters and compare performance to make a decision on the best hyperparameter values for our model.

Choosing the best number of clusters

The weakness of k-means clustering is that we don't know how many clusters we need by just running the model. We need to test ranges of values and make a decision on the best value of k . We typically make a decision using the Elbow method to determine the optimal number of clusters where we are both not overfitting the data with too many clusters, and also not underfitting with too few.

We create the below loop to test and store different model results so that we can make a decision on the best number of clusters.

```
range(2, 8)
= []
e = []

k in K:
    # train the model for current value of k on training data
    model = KMeans(n_clusters = k, random_state = 0, n_init='auto').fit(X_train_norm)

    # append the model to fits
    fits.append(model)

    # Append the silhouette score to scores
    score.append(silhouette_score(X_train_norm, model.labels_, metric='euclidean'))
```

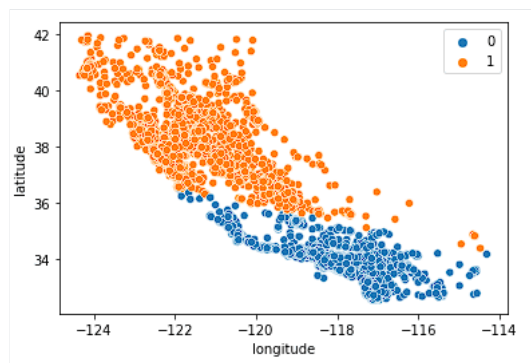

[🔗 Explain code](#)
POWERED BY  datalab

We can then first visually look at a few different values of k .

First we look at $k = 2$.

```
atplotlib(data = X_train, x = 'longitude', y = 'latitude', hue = fits[0].label)
```

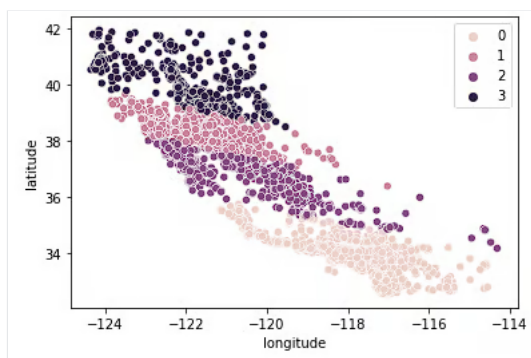


[✈ Explain code](#)
POWERED BY  datacamp

The model does an ok job of splitting the state into two halves, but probably doesn't capture enough nuance in the California housing market.

Next, we look at $k = 4$.

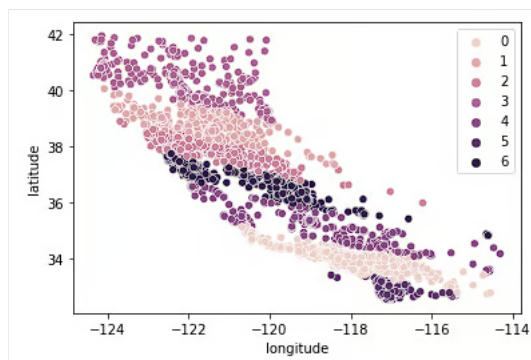
```
sns.scatterplot(data = X_train, x = 'longitude', y = 'latitude', hue = fits)
```

[✈ Explain code](#)
POWERED BY  datacamp

We see this plot groups California into more logical clusters across the state based on how far North or South the houses are in the state. This model most likely captures more nuance in the housing market as we move across the state.

Finally, we look at $k = 7$.

```
sns.scatterplot(data = X_train, x = 'longitude', y = 'latitude', hue = fits)
```

[✈ Explain code](#)
POWERED BY  datacamp

The above graph appears to have too many clusters. We have sacrifice easy interpretation of the clusters for a “more accurate” geo-clustering result.

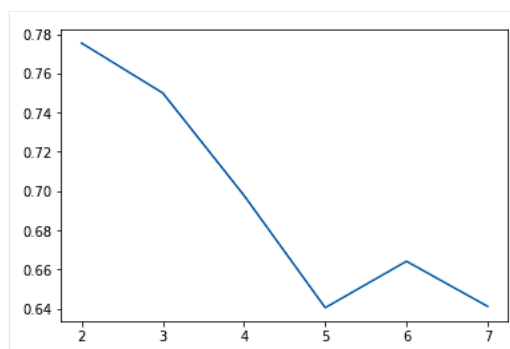
Typically, as we increase the value of K , we see improvements in clusters and what they represent until a certain point. We then start to see diminishing returns or even worse

performance. We can visually see this to help make a decision on the value of k by using an elbow plot where the y-axis is a measure of goodness of fit and the x-axis is the value of k .

```
sns.lineplot(x = K, y = score)
```


[✦ Explain code](#)

POWERED BY datalab



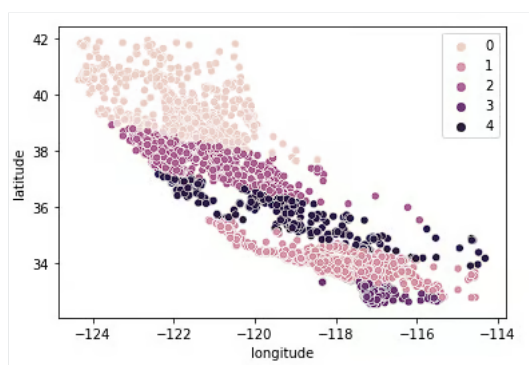
We typically choose the point where the improvements in performance start to flatten or get worse. We see $k = 5$ is probably the best we can do without overfitting.

We can also see that the clusters do a relatively good job of breaking California into distinct clusters and these clusters map relatively well to different price ranges as seen below.

```
sns.scatterplot(data = X_train, x = 'longitude', y = 'latitude', hue = fits['label'])
```


[✦ Explain code](#)

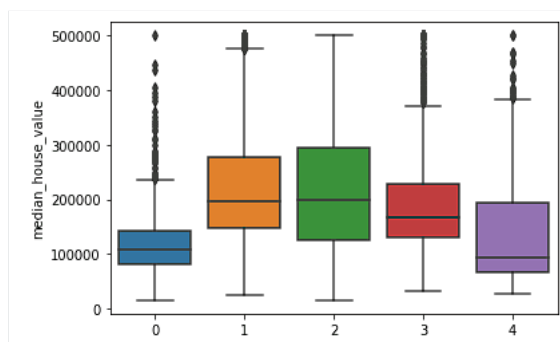
POWERED BY datalab



```
sns.boxplot(x = fits['label'], y = y_train['median_house_value'])
```


[✦ Explain code](#)

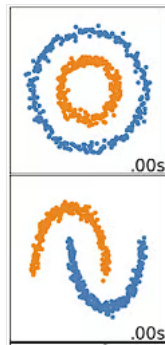
POWERED BY datalab



When will k-means cluster analysis fail?

K-means clustering performs best on data that are spherical. Spherical data are data that group in space in close proximity to each other either. This can be visualized in 2 or 3

dimensional space more easily. Data that aren't spherical or should not be spherical do not work well with k-means clustering. For example, k-means clustering would not do well on the below data as we would not be able to find distinct centroids to cluster the two circles or arcs differently, despite them clearly visually being two distinct circles and arcs that should be labeled as such.



[Image Source](#)

There are many other clustering algorithms that do a good job of clustering non-spherical data, covered in [Clustering in Machine Learning: 5 Essential Clustering Algorithms](#).

Should you split your data into training and testing sets?

The decision to split your data depends on what your goals are for clustering. If the goal is to cluster your data as the end of your analysis, then it is not necessary. If you are using the clusters as a feature in a supervised learning model or for prediction (like we do in the [Scikit-Learn Tutorial: Baseball Analytics Pt 1](#) tutorial), then you will need to split your data before clustering to ensure you are following best practices for the supervised learning workflow.

Take it to the Next Level

Now that we have covered the basics of k-means clustering in Python, you can check out this [Unsupervised Learning in Python](#) course for a good introduction to k-means and other unsupervised learning algorithms. Our more advanced course, [Cluster Analysis in Python](#), gives a more in-depth look at clustering algorithms and how to build and tune them in Python. Finally, you can also check out the [An Introduction to Hierarchical Clustering in Python](#) tutorial as an approach which uses an alternative algorithm to create hierarchies from data.

Earn a Top AI Certification

Demonstrate you can effectively and responsibly use AI.

Get Certified, Get Hired

Top Courses on Machine Learning

Cluster Analysis in Python

Beginner ⌚ 4 hr 👥 56.1K learners

In this course, you will be introduced to unsupervised learning through techniques such as hierarchical and k-means clustering using the SciPy library.

[See Details](#) →

Unsupervised Learning in Python

Beginner ⌚ 4 hr 👤 146.1K learners

Learn how to cluster, transform, visualize, and extract insights from unlabeled datasets using scikit-learn and scipy.

[See Details →](#)

[See More →](#)

TOPICS

[Machine Learning](#) [Data Science](#) [Python](#)

Learn more about Machine Learning

COURSE

Ensemble Methods in Python

⌚ 4 hr 👤 9.8K

Learn how to build advanced and effective machine learning models in Python using ensemble techniques such as bagging, boosting, and stacking.

[See Details →](#)

[Start Course](#)

[See More →](#)

Related



TUTORIAL

An Introduction to Hierarchical Clustering in Python



TUTORIAL

Introduction to Machine Learning in Python



TUTORIAL

K-Means Clustering in R Tutorial

[See More →](#)

Grow your data skills with DataCamp for Mobile

Make progress on the go with our mobile courses and daily 5-minute coding challenges.



LEARN

[Learn Python](#)

[Learn R](#)

[Learn AI](#)

[Learn SQL](#)

[Learn Power BI](#)

[Learn Tableau](#)

[Learn Data Engineering](#)

[Assessments](#)

[Career Tracks](#)

[Skill Tracks](#)

[Courses](#)

[Data Science Roadmap](#)

DATA COURSES

[Python Courses](#)

[R Courses](#)

[SQL Courses](#)

[Power BI Courses](#)

[Tableau Courses](#)

[Alteryx Courses](#)

[Azure Courses](#)

[Google Sheets Courses](#)

[AI Courses](#)

[Data Analysis Courses](#)

[Data Visualization Courses](#)

[Machine Learning Courses](#)

[Data Engineering Courses](#)

[Probability & Statistics Courses](#)

DATALAB

[Get Started](#)

[Pricing](#)

[Security](#)

[Documentation](#)

CERTIFICATION

Certifications

Data Scientist

Data Analyst

Data Engineer

SQL Associate

Power BI Data Analyst

Tableau Certified Data Analyst

Azure Fundamentals

AI Fundamentals

RESOURCES

Resource Center

Upcoming Events

Blog

Code-Alongs

Tutorials

Docs

Open Source

RDocumentation

Course Editor

Book a Demo with DataCamp for Business

Data Portfolio

Portfolio Leaderboard

PLANS

Pricing

For Business

For Universities

Discounts, Promos & Sales

DataCamp Donates

FOR BUSINESS

Business Pricing

Teams Plan

Data & AI Unlimited Plan

Customer Stories

Partner Program

ABOUT

[About Us](#)[Learner Stories](#)[Careers](#)[Become an Instructor](#)[Press](#)[Leadership](#)[Contact Us](#)[DataCamp Español](#)[DataCamp Português](#)[DataCamp Deutsch](#)[DataCamp Français](#)

SUPPORT

[Help Center](#)[Become an Affiliate](#)[Privacy Policy](#) [Cookie Notice](#) [Do Not Sell My Personal Information](#) [Accessibility](#) [Security](#) [Terms of Use](#)

© 2024 DataCamp, Inc. All Rights Reserved.

[TUTORIALS](#) [Categories](#)  

EN